

Module 2:

Project Structure & Stages

Learning Agenda

- By the end of this module, learners will:
 - Understand the CDK project folder layout and how stacks fit into it
 - Configure and manage multiple environments (dev/stage/prod)
 - Apply environment-aware design (“synth once, deploy many”)
 - Distinguish stateless vs stateful stacks
 - Build stacks using stack props
 - Use cross-stack references and outputs correctly

CDK Project Layout

- `cdk.json` → CDK app entry + context
- `bin/` → App entrypoint (creates stacks)
- `lib/` → Stack definitions
- `node_modules/` → Dependencies
- `cdk.out/` → Synthesized CloudFormation templates



Multi-Environment CDK (dev/stage/prod)

- Same codebase → multiple deployments
- Different accounts/regions per environment
- Defined in bin/<app>.ts
- Stacks are created conditionally per environment

Example:

- `new MyAppStack(app, 'MyApp-Dev', { env: devEnv });`
- `new MyAppStack(app, 'MyApp-Stage', { env: stageEnv });`
- `new MyAppStack(app, 'MyApp-Prod', { env: prodEnv });`

Synth Once, Deploy Many

- One codebase
- One synthesized template
- Deployed multiple times
- Reduces drift, improves auditability

CloudFormation Template → Dev | Stage | Prod

Stateless vs Stateful Stacks

- Stateless: can be created/destroyed anytime
 - API Gateway, Lambda, IAM roles, ECS services
- Stateful: should not be recreated often
 - S3 buckets, DynamoDB tables, RDS, VPCs

Stack Props & Env-Aware Config

```
interface MyStackProps extends StackProps {  
    environment: string; // dev | stage | prod  
}  
  
new MyStack(app, 'MyApp', {  
    environment: 'dev',  
});
```

Cross-Stack References & Outputs

- **Why use them?**
 - Share VPC ID between stacks
 - Share DynamoDB table names
 - Reduce duplication & configuration drift

Example:

```
const vpc = new VpcStack(app, 'VpcStack');  
new AppStack(app, 'AppStack', {  
  vpc: vpc.vpc,  
});
```


Questions?